

Practical - 6

AIM: There are n servers numbered from 0 to $n - 1$ connected by undirected server-to-server connections forming a network where $\text{connections}[i] = [a_i, b_i]$ represents a connection between servers a_i and b_i . Any server can reach other servers directly or indirectly through the network. A critical connection is a connection that, if removed, will make some servers unable to reach some other servers. Return all critical connections in the network in any order.

PROGRAM CODE :-

```
#include <stdio.h> #include
<stdlib.h>

typedef struct {
    int u, v;
} Edge;

typedef struct {
    int *arr;
    int size;
    int capacity;
} IntArray;

void initArray(IntArray *a, int initialSize) {
    a->arr = (int *)malloc(initialSize * sizeof(int));a-
    >size = 0;
    a->capacity = initialSize;
```

```
}
```

```
void insertArray(IntArray *a, int element) {if
```

```
    (a->size == a->capacity) {
```

```
        a->capacity *= 2;
```

```
        a->arr = (int *)realloc(a->arr, a->capacity * sizeof(int));
```

```
    }
```

```
    a->arr[a->size++] = element;
```

```
}
```

```
void freeArray(IntArray *a) {
```

```
    free(a->arr);
```

```
    a->arr = NULL;
```

```
    a->size = a->capacity = 0;
```

```
}
```

```
typedef struct {
```

```
    Edge *arr; int
```

```
    size;
```

```
    int capacity;
```

```
} EdgeArray;
```

```
void initEdgeArray(EdgeArray *a, int initialSize) {
```

```
    a->arr = (Edge *)malloc(initialSize * sizeof(Edge));
```

```
a->size = 0;

a->capacity = initialSize;

}

void insertEdgeArray(EdgeArray *a, int u, int v) {if
    (a->size == a->capacity) {
        a->capacity *= 2;
        a->arr = (Edge *)realloc(a->arr, a->capacity * sizeof(Edge));

    }
    a->arr[a->size].u = u;
    a->arr[a->size].v = v;
    a->size++;
}

void freeEdgeArray(EdgeArray *a) {
    free(a->arr);
    a->arr = NULL;
    a->size = a->capacity = 0;
}

void dfs(int node, int parent, IntArray *graph, int *ids, int *low, int
    *visited, EdgeArray *bridges, int *time) {
    visited[node] = 1;
    ids[node] = low[node] = (*time)++;

    for (int i = 0; i < graph[node].size; i++) {
```

```
int neighbor = graph[node].arr[i]; if
(neighbor == parent) continue;

if (!visited[neighbor]) {
    dfs(neighbor, node, graph, ids, low, visited, bridges, time); low[node]
    = (low[node] < low[neighbor]) ? low[node] :
low[neighbor];
    if (ids[node] < low[neighbor]) { insertEdgeArray(bridges,
        node, neighbor);
    }
} else {
    low[node] = (low[node] < ids[neighbor]) ? low[node] : ids[neighbor];
}
}
}
```

```
EdgeArray criticalConnections(int n, Edge connections[], int m) {
    IntArray *graph = (IntArray *) malloc(n * sizeof(IntArray)); for
    (int i = 0; i < n; i++) {
        initArray(&graph[i], 4); // Assume initial size of 4 for each adjacency
list
    }

    for (int i = 0; i < m; i++) { int
        u = connections[i].u;
```

```
int v = connections[i].v;

insertArray(&graph[u], v);

insertArray(&graph[v], u);

}


int *ids = (int *)malloc(n * sizeof(int)); int
*low = (int *)malloc(n * sizeof(int)); int
*visited = (int *)calloc(n, sizeof(int));

EdgeArray bridges;

initEdgeArray(&bridges, 1);

int time = 0;

for (int i = 0; i < n; i++) {
    if (!visited[i]) {
        dfs(i, -1, graph, ids, low, visited, &bridges, &time);
    }
}


for (int i = 0; i < n; i++) {
    freeArray(&graph[i]);
}

free(graph);

free(ids);

free(low);

free(visited);
```

```
        return bridges;
    }

int main() {int
    n, m;

    printf("Enter the number of servers (n): ");
    scanf("%d", &n);

    printf("Enter the number of connections (m): ");
    scanf("%d", &m);

    Edge *connections = (Edge *)malloc(m * sizeof(Edge));

    printf("Enter the connections (each connection as two space-separated
    integers):\n");

    for (int i = 0; i < m; i++) {
        scanf("%d %d", &connections[i].u, &connections[i].v);
    }

    EdgeArray criticalConnectionsList = criticalConnections(n,
    connections, m);

    printf("Critical connections in the network are:\n");for (int
    i = 0; i < criticalConnectionsList.size; i++) {
        printf("%d %d\n", criticalConnectionsList.arr[i].u,
    criticalConnectionsList.arr[i].v);
    }
```

```
free(connections);  
  
freeEdgeArray(&criticalConnectionsList);  
  
return 0;  
  
}
```

OUTPUT :-

Enter the number of servers (n): 6 Enter

the number of connections (m): 7

Enter the connections (each connection as two space-separated integers): 0 1

1 2

2 0

1 3

3 4

4 5

3 5

Critical connections in the network are: 1 3

```
main.c
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct {
5     int u, v;
6 } Edge;
7
8 typedef struct {
9     int *arr;
10    int size;
11    int capacity;
12 } IntArray;
13
14 void initArray(IntArray *a, int initialSize) {
15     a->arr = (int *)malloc(initialSize * sizeof(int));
16     a->size = 0;
17     a->capacity = initialSize;
18 }
19
20 void insertArray(IntArray *a, int element) {
21     if (a->size == a->capacity) {
22         a->capacity *= 2;
23         a->arr = (int *)realloc(a->arr, a->capacity * sizeof(int));
24     }
25     a->arr[a->size++] = element;
26 }
27
28 void freeArray(IntArray *a) {
29     free(a->arr);
30 }
```

```

Enter the number of servers (N): 4
Enter the number of connections (M): 7
Enter the connections (each connection as two space-separated integers):
0 1
1 2
2 0
1 3
3 4
4 5
3 5
Critical connections in the network are:
1 3

=== Code Execution Successful ===

```